

Relatório de Laboratório

Laboratório de Experimentação de Software — Modelo/Template de Relatório

Este documento é um MODELO válido para qualquer um dos 5 laboratórios da disciplina (Lab01 a Lab05). Os parágrafos em itálico cinza/verde, iniciados por “ORIENTAÇÃO”, explicam o que cada subseção deve conter — apague-os e escreva o conteúdo real do grupo no lugar. Textos entre colchetes [assim] indicam onde inserir informação específica do seu grupo/laboratório.

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	[Lab01 — Características de repositórios populares]
Grupo (trio)	Pedro Luis · Enrico Bessa · Pericles Pires
Link do repositório / GitHub Projects	https://github.com/PedroL10/Lab01Exp
Data de entrega	20/08/2026

1. Introdução

Repositórios populares no GitHub costumam ser usados como referência implícita de qualidade e boas práticas de engenharia de software, mas nem sempre fica claro quais características eles de fato compartilham. Este laboratório investiga essa questão de forma empírica, coletando métricas dos 1.000 repositórios com mais estrelas do GitHub via API GraphQL.

Perguntas que esta seção deve responder ao leitor

- Qual problema está sendo investigado, e por que ele importa (para a engenharia de software, para o mercado, ou para o próprio grupo)?
- Quais são as Questões de Pesquisa do enunciado (numeradas RQ1, RQ2, ...)?
- Quais as hipóteses informais do grupo para cada RQ, antes de olhar os dados (quando aplicável ao laboratório)?
- Quais RQs, métricas ou variáveis o grupo está propondo além do enunciado (resumo de 1 linha cada — os 30% de inovação)?
- =====
- RQ01 - Sistemas populares são maduros/antigos?
- RQ02 - Sistemas populares recebem muita contribuição externa?
- RQ03 — Sistemas populares lançam releases com frequência?
- RQ04 — Sistemas populares são atualizados com frequência?
- RQ05 — Sistemas populares são escritos nas linguagens mais populares?
- RQ06 — Sistemas populares possuem alto percentual de issues fechadas?

[conteúdo do grupo — substituir este texto]

2. Contexto

Este é o Lab01 da disciplina, primeiro laboratório do semestre — não depende de dados de laboratórios anteriores. O objeto de estudo são os 1.000 repositórios com mais estrelas do GitHub, coletados via API GraphQL oficial

3. Metodologia

ORIENTAÇÃO: Esta é a seção mais longa do relatório e a que mais evidencia o trabalho real do grupo. Ela tem seis subseções — as cinco primeiras cobrem principalmente os 70% do enunciado; a última (Inovações) é onde os 30% de contribuição própria do grupo devem ficar explícitos e fáceis de identificar na correção.

3.1 Principais Desafios

ORIENTAÇÃO: Relate as dificuldades técnicas e metodológicas reais enfrentadas pelo grupo — não uma lista de trivialidades já resolvidas, e sim decisões difíceis de fato. Exemplos típicos, conforme o laboratório: limite

de taxa (rate limit) da API do GitHub ao consultar milhares de repositórios ou workflow runs (Lab01/Lab03); paginação de grandes volumes de dados; ausência de histórico de mudança de status consultável via API no GitHub Projects, exigindo snapshots manuais recorrentes (todos os laboratórios); dificuldade de padronizar katas de dificuldade equivalente e evitar memorização de soluções pela IA (Lab02); ambiguidade na definição operacional de uma métrica, como lead time (Lab03); dados incompletos ou repositórios sem GitHub Actions habilitado (Lab03) -

=====

Custo de processamento da API do GitHub: solicitar muitos repositórios de uma vez (50-100) com campos aninhados (issues, releases, pull requests) excedia o limite de custo da API e retornava erro 502/504, provavelmente por causa de repositórios com trackers de issues/PRs muito grandes. Resolvido com paginação em lotes pequenos (10 repositórios por requisição) e retry com backoff exponencial para erros 502/503/504.

- Limite de 1.000 resultados da Search API do GitHub: tanto a API REST quanto a GraphQL limitam buscas a no máximo 1.000 resultados totais, independente da paginação — confirmado na prática ao coletar exatamente 1.000 repositórios sem erro.

- Certificado SSL na rede da instituição: pip install e as chamadas à API falhavam por erro de certificado SSL não confiável na rede do laboratório. Resolvido instalando pip-system-certs, que faz o Python usar o repositório de certificados do próprio Windows.

- Divergência entre fontes de contagem de PRs aceitas: ao validar manualmente pull_requests_merged_total via API REST de busca (/search/issues), os valores ficaram levemente diferentes (menos de 1,5%) dos obtidos pela query GraphQL (pullRequests(states: MERGED)). Explicado pela API de busca usar um índice eventualmente consistente, enquanto a query GraphQL consulta a fonte primária diretamente — por isso o projeto usa a GraphQL como fonte oficial.

- Ausência de histórico de status no GitHub Projects: exigiu snapshots manuais recorrentes via script GraphQL (ver Issue de snapshot), já que a API não expõe histórico de mudança de coluna.

3.2 Tomadas de Decisão

ORIENTAÇÃO: *Documente as decisões metodológicas do grupo e o raciocínio (trade-off) por trás de cada uma — não apenas a escolha final. Exemplos que os enunciados pedem explicitamente: o limite de WIP definido para a coluna Doing e sua justificativa (obrigatório em todo laboratório); qual assistente de IA foi usado e por quê, e como se garantiu o mesmo tratamento em todos os trials (Lab02); qual definição operacional de métrica foi adotada quando o enunciado permite variação, mantendo-a consistente para toda a amostra (ex.: lead time no Lab03); critério de inclusão/exclusão de repositórios na amostra; linguagem de programação escolhida em função da ferramenta de métricas estáticas disponível (CK exige Java; Radon para Python).*

=====

- Linguagem do projeto: Python, escolhido pela boa integração com análise de dados nas próximas sprints (RQs 03-07, visualizações).

- Fonte primária de PRs aceitas: GraphQL (`pullRequests(states: MERGED) { totalCount }`) em vez da API de busca, por ser mais confiável (ver 3.1).
- Tamanho de lote (page size) na paginação: fixado em 10 repositórios por requisição — testado empiricamente; lotes maiores (25-100) causavam erro de custo da API em repositórios com trackers muito grandes.
- Critério de amostra: `search(query: "stars:>1 sort:stars-desc", type: REPOSITORY)` — exclui repositórios com 0-1 estrela (ruído) e ordena por estrelas decrescente.
- Limite de WIP na coluna Doing: 3 cartões — um por integrante ativo no board (trio), garantindo que cada pessoa tenha no máximo uma tarefa em andamento por vez, evitando dispersão de foco e cartões abertos sem responsável dedicado simultâneo.
- Formato de saída por sprint: JSON na Sprint 1 (100 repositórios, inspeção/validação), CSV na Sprint 2 (1.000 repositórios, formato exigido pelo enunciado para a base final).

[conteúdo do grupo — substituir este texto]

3.3 Etapas

ORIENTAÇÃO: Descreva o processo de desenvolvimento em sprints, seguindo a estrutura do enunciado (ex.: Lab0XS01, S02, S03 + Relatório Final), com o que foi efetivamente entregue em cada uma e quem (qual integrante) foi responsável por qual parte — a correção do professor é feita a partir do board (GitHub Projects), então a divisão aqui deve refletir os Assignees reais das Issues, não uma divisão apenas narrativa. Inclua também a subseção “Configuração do processo” exigida em todos os laboratórios: as colunas do board (mínimo Backlog → To Do → Doing → Review → Done), a política de limite de WIP em uso, e uma captura de tela (print) do board ao final do laboratório, mostrando o fluxo real de trabalho do grupo.

Sprint S01:

- Autenticação GraphQL configurada — Responsável: Pedro Luis— Issue #1
- Query base para 100 repositórios — Responsável: Pedro Luis— Issue #2
- Validação de campos RQ01+RQ02 (amostra) — Responsável: Pedro Luis— Issue #3
- Validação de campos RQ03+RQ04 (amostra) — Responsável: Enrico Bessa — Issue #4 [PENDENTE]
- Validação de campos RQ05+RQ06 (amostra) — Responsável: Pericles Pires — Issue #5 [PENDENTE]
- Integração da coleta final (100 repositórios) + registro em JSON — Responsável: Pedro Luis— Issues #6 e #7

Sprint S02:

- Paginação + exportação de 1.000 repositórios em CSV — Responsável: Pedro Luis— Issue #8
- Validação RQ01+RQ02 (1.000 repositórios) + hipótese informal — Responsável: Pedro Luis— Issue #10

- Validação RQ03+RQ04 (1.000 repositórios) + hipótese informal — Responsável: Integrante B — Issue [PREENCHER Nº] [PENDENTE]

- Validação RQ05+RQ06 (1.000 repositórios) + hipótese informal — Responsável: Integrante C — Issue [PREENCHER Nº] [PENDENTE]

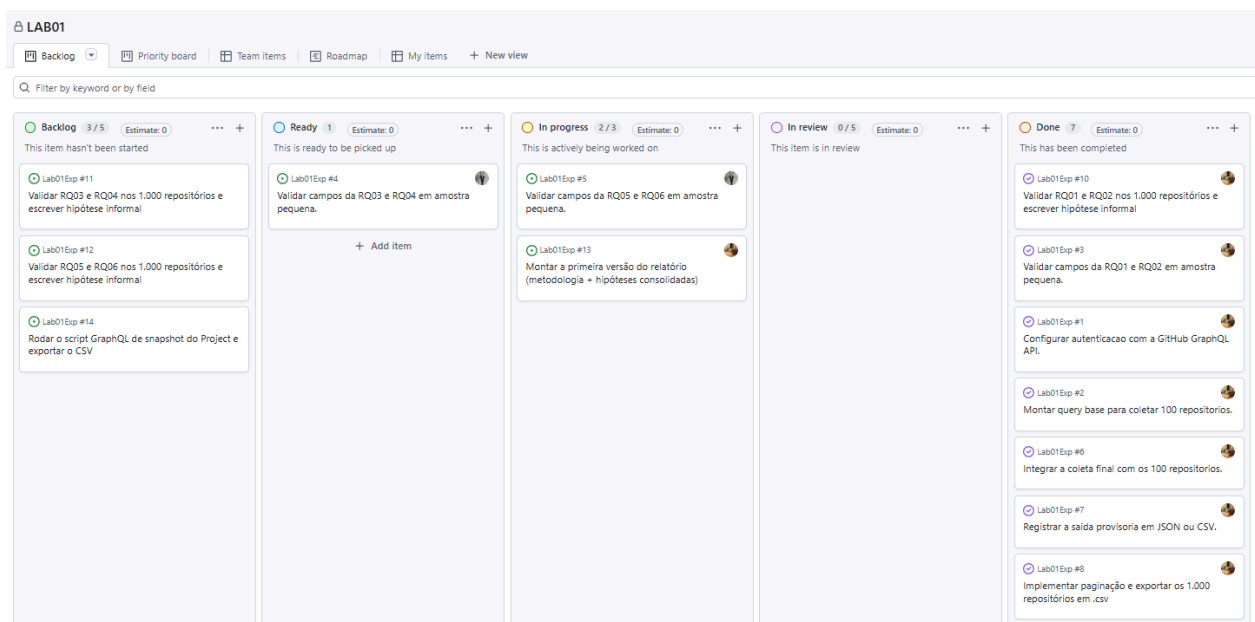
- Primeira versão do relatório — Responsável: Pedro Luis — Issue [PREENCHER Nº 13]

- Snapshot do Project exportado em CSV — Responsável: Você — Issue [PREENCHER Nº]

Configuração do processo:

- Colunas do board: Backlog → To Do → Doing → Review → Done.

- Limite de WIP em Doing: 3 cartões (um por integrante ativo no board).



[Tabela ou linha do tempo com Sprint | Entregas | Responsável(is) | Issues (nº)]

Sugestão: insira aqui o print do quadro Kanban (GitHub Projects) mencionado na orientação acima.

3.4 Ferramentas

ORIENTAÇÃO: Liste as ferramentas usadas na coleta, processamento e análise de dados — sejam específicas (nome e versão quando relevante), não genéricas. Exemplos conforme o laboratório: GraphQL e/ou REST API do GitHub para mineração (Lab01/Lab03 — bibliotecas de terceiros para consulta à API não são permitidas, o script deve ser próprio do grupo); Python/Pandas para manipulação de dados; Matplotlib/Seaborn ou Plotly/Dash/Streamlit para visualização; CK, PMD ou Radon para métricas estáticas de código (Lab02);

testes estatísticos como o de Wilcoxon para amostras pareadas (Lab02); ferramenta de BI (Power BI, Tableau, Looker Studio) caso o grupo não opte pelo dashboard em código (Lab04). Inclua também a ferramenta de processo, obrigatória em todos os laboratórios: GitHub Projects (v2), com o link do repositório/board do grupo.

3.4 Ferramentas

- Python 3.11
- requests (cliente HTTP genérico, consumindo a query GraphQL escrita pelo próprio grupo)
- python-dotenv (variáveis de ambiente)
- pip-system-certs (compatibilidade de certificados SSL)
- API GraphQL oficial do GitHub (<https://api.github.com/graphql>)
- GitHub Projects (v2) — <https://github.com/users/PedroL10/projects/1/views/1>

[conteúdo do grupo — substituir este texto]

3.5 Tabela de Métricas

ORIENTAÇÃO: Construa uma tabela relacionando cada Questão de Pesquisa à métrica correspondente, sua definição operacional exata (a fórmula ou regra de cálculo — não basta o nome) e a ferramenta/fonte usada para coletá-la. Isso é o que garante que o laboratório seja reprodutível por outro grupo. A primeira linha abaixo é um exemplo ilustrativo (baseado no Lab01); substitua pelas RQs e métricas do seu laboratório.

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Data atual – data de criação do repositório	Dias/anos	Script GraphQL (API do GitHub)
RQ02	Total de PRs aceitas	<code>pullRequests(states: MERGED) { totalCount }</code>	quantidade/contagem	Script GraphQL (API do GitHub)

3.6 Inovações Propostas pelo Grupo (30% da nota)

ORIENTAÇÃO: O enunciado do laboratório corresponde a 70% da exigência da disciplina. Os outros 30% dependem de uma contribuição original do grupo, que deve estar claramente identificada aqui — não diluída no restante do texto — para facilitar a correção. Escolha uma ou mais frentes de inovação, entre: (a) uma nova Questão de Pesquisa, além das do enunciado; (b) uma métrica ou variável adicional, não pedida no enunciado; (c) uma mudança de arquitetura/ferramenta de coleta (ex.: paralelizar a coleta, usar cache, trocar de biblioteca de visualização); (d) uma metodologia alternativa ou complementar (ex.: um teste estatístico adicional, uma segmentação diferente da amostra, uma técnica de controle de ameaça à validade não exigida pelo enunciado). Para cada inovação escolhida, explique o que foi feito, por que o grupo considerou relevante, e onde o resultado dela aparece nas seções de Resultados/Discussão e na Conclusão — inovação sem resultado discutido não conta como contribuição efetiva.

[conteúdo do grupo — substituir este texto]

4. Resultados

4.1 Coleta de Dados

ORIENTAÇÃO: Relate o volume final de dados efetivamente coletado e analisado — não apenas o volume-alvo do enunciado. Informe: quantos itens restaram após os filtros de qualidade (ex.: dos 1.000 repositórios buscados, quantos tinham dados completos; dos repositórios candidatos, quantos de fato usavam GitHub Actions — Lab03); o período coberto pela coleta; quantos trials/execuções foram concluídos dentro do tempo (Lab02); quantos snapshots do Kanban estão disponíveis e desde quando (Lab04/Lab05); outliers ou dados ausentes identificados, e como foram tratados (removidos, mantidos e discutidos à parte, etc.).

[conteúdo do grupo — substituir este texto]

4.2 Visualização Gráfica

ORIENTAÇÃO: Para cada Questão de Pesquisa (do enunciado e das RQs de inovação do grupo), inclua ao menos uma visualização que a responda diretamente, com a pergunta enunciada em texto imediatamente antes do gráfico correspondente, eixos nomeados com clareza e a medida de tendência central adequada indicada (mediana costuma ser preferível a média quando há outliers ou distribuição assimétrica — comum em dados de repositórios de software). Use o tipo de gráfico adequado ao tipo de pergunta, conforme a tabela abaixo, e explicita no texto os valores-chave que aparecem no gráfico (não deixe o leitor “adivinhar” o número a partir da figura).

Tipo de pergunta / dado	Gráfico recomendado
Comparar uma métrica entre categorias (ex.: linguagem, benchmark DORA)	Barras (ranking) — ordenadas por valor, não alfabeticamente
Comparar dois tratamentos no mesmo grupo (ex.: com IA vs. sem IA)	Boxplot pareado ou gráfico de pontos conectados (before/after)
Distribuição de uma métrica numérica (ex.: idade dos repositórios)	Histograma ou boxplot
Relação entre duas métricas numéricas (ex.: RQ07 do Lab01, RQ05 do Lab03)	Gráfico de dispersão (scatter plot)
Evolução ao longo do tempo (ex.: cycle time por sprint)	Linha, com um ponto por sprint/snapshot

Composição/fluxo do Kanban ao longo do tempo (Cumulative Flow Diagram)	Área empilhada (uma camada por coluna do board)
Proporção de categorias (ex.: % de issues fechadas)	Barra única 100% ou barras simples — evite pizza com muitas fatias

[Insira aqui os gráficos do grupo, um por RQ, cada um precedido da pergunta que ele responde]

4.3 Discussão

ORIENTAÇÃO: Para cada RQ (do enunciado e das RQs de inovação do grupo), compare explicitamente a hipótese informal levantada na Introdução com o resultado efetivamente obtido — hipótese confirmada, refutada, ou parcialmente confirmada, e por quê. Quando houver teste estatístico (ex.: Wilcoxon no Lab02), reporte o valor obtido e interprete o que ele significa em linguagem acessível, não apenas o número bruto. Discuta as ameaças à validade específicas do laboratório (ex.: efeito de aprendizado entre katas e risco de memorização pela IA — Lab02; diferença de dificuldade entre laboratórios distintos confundindo a tendência de cycle time — Lab05; lacunas nos snapshots do Kanban — Lab05). Finalize relacionando o que as inovações do grupo (seção 3.6) acrescentaram: elas confirmaram, contradisseram ou aprofundaram o que os 70% do enunciado já mostravam?

[conteúdo do grupo — substituir este texto]

5. Conclusão

ORIENTAÇÃO: Sintetize, em poucos parágrafos, as respostas a todas as RQs (enunciado + inovação do grupo), sem repetir números já discutidos em detalhe — o objetivo aqui é a mensagem final, não os dados brutos. Aponte as principais limitações do estudo (tamanho de amostra, ameaças à validade não mitigadas, período de coleta). Quando o enunciado pedir explicitamente uma postura de consultoria (caso do Lab05, que pede recomendações de melhoria de processo “como se o grupo fosse consultoria para um time real”), inclua recomendações objetivas e acionáveis, não genéricas. Encerre indicando o que o grupo faria diferente com mais tempo ou recursos, e quais das inovações propostas (30%) valeriam a pena expandir em um trabalho futuro.

[conteúdo do grupo — substituir este texto]

5. Referências

- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.
-